20 January 2026 21:47

Assessment Task

1. Rank the following functions by their asymptotic growth rate from slowest to fastest. For each pair in your ranking, provide a brief mathematical justification: n^2, $n \log n$, 2^n, $n!$, $\log(n!)$, $n^{1.5}$.
2. Using the formal definition of O-notation, prove that $f(n) = 5n^3 + 2n^2 + 20$ is $O(n^3)$. You must explicitly identify the constants c and n_0 that satisfy the condition.
3. Consider a standard linear search algorithm. State a formal loop invariant for the search process and use it to prove the correctness of the algorithm (Initialization, Maintenance, and Termination).
4. Use the Master Method to solve the following recurrence: $T(n) = 4T(n/2) + n^2$. Show your step-by-step comparison of $f(n)$ with $n^{\log_b a}$.
5. Analyze the asymptotic time complexity of the following pseudo-code. Provide the result in Big-O notation and show the summation used to derive it:

```
for i = 1 to n:
    for j = 1 to i * i:
        print(i, j)
```

6. The "Change-Making Problem" seeks to use the minimum number of coins for a value V. Provide a set of coin denominations where the greedy approach (always taking the largest coin first) fails to give the optimal solution. Show the step-by-step process of the greedy failure versus the optimal result.
7. Explain why any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. Focus on the relationship between the height of a decision tree and the number of reachable leaves.
8. Define what it means for a sorting algorithm to be "stable." Provide a trace (a step-by-step sequence) of an unstable sort (like Heapsort or Quicksort) on a small array with duplicate keys to demonstrate how the relative order of equal elements can change.
9. Briefly define the classes **P** and **NP**. Why is a problem in **NP** not necessarily "hard," and what role does "Polynomial-time Verifiability" play in this definition?
10. Research the Counting Sort algorithm. Explain the process and describe the specific scenario (regarding the range of input values k) where this algorithm is preferred over $O(n \log n)$ sorting algorithms.
11. Explain the "Potential Method" of amortized analysis. How does this differ from simple "average-case" analysis? Use the process of incrementing a binary counter as a conceptual example.

1. let $c=1$ and $n_0=1$
for all $n \geq n_0$ $n^{1.5} \leq n^2$ so $n^{1.5} \in O(n^2)$

$\lim_{n\to\infty} \frac{n^{1.5}}{n\log n} = \lim_{n\to\infty} \frac{n^{0.5}}{\log n} = \frac{0.5n^{-0.5}}{\frac{1}{n}} = 0.5n^{0.5} = \infty$

(L'Hopital's rule) $\Rightarrow n^{1.5} \in \omega(n\log n)$

$\log(n!) = \log 1 + \log 2 + \dots + \log n < n\log n$ / $\log(n!) \geq \frac{n}{2}\log\frac{n}{2} - \frac{n}{2}\log 2 \geq \frac{n}{4}\log n$

$\log(n!) \in \Theta(n\log n)$ $\log(n!)$? $n\log n < n^{1.5} < n^2 < 2^n < n!$

$n! = 1\cdot 2\cdot 3 \cdot \dots \cdot n > 1 \cdot \overbrace{2\cdot 2\cdot 2 \cdots 2}^{n-1 \text{ terms}}$

$n! > 1 \cdot 2^{n-1}$

$n! > \frac{1}{2}\cdot 2^n$ $n! \in \Omega(2^n)$

n^2	2^n
$2\log n$	$n\log 2$
$\frac{2}{\log 2}\log n$	n

$\frac{2}{\log 2}\lim \frac{\log n}{n} = \frac{1}{n} = 0 \Leftrightarrow \log n \in o(n) \Leftrightarrow n^2 \in o(2^n)$

2. $\exists c, n_0$, s.t. $cn^3 \geq f(n) \geq 0$

let $c=6$ $6n^3 = 5n^3+2n^2+20$

$0 = -n^3+2n^2+20$

$\max(n_1, n_2, n_3) = 1+\sqrt{21} \approx 5.58$

$c=6$

let $n_0 = 6$, $\forall n \geq n_0$, $6n^3 \geq f(n) \geq 0$

$\therefore f(n) \in O(n^3)$

3. 不会 (loop invariant?) properly each iteration holds

4. 不会 (Master method?)

5. $\sum_{i=1}^{n}\sum_{j=1}^{i^2} j = 1 + (1+2+3+\overset{2^2}{4}) + (4 \dots + \overset{3^2}{9}) + \dots + (4 \dots + n^2)$

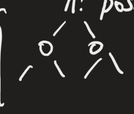
$\Downarrow$

$\sum_{1}^{n^2} k = \frac{n^2(n^2+1)}{2} = \frac{n^4+n^2}{2}$

let $T(n)$ be the needed, $T(n) \in O(n^4)$

6. 不会

7. $\log(n!)$ n! possible sequence



$O(\log(n!)) \in \Theta(n\log n) \subseteq O(n\log n)$

8. Stable sort, duplicate elements remain in the same order after sorting. ×

9. 不会

10. count the frequency of the value and output according to the frequency

$O(N)$, range of values must be small / dense

space → the range of values.

11. 不会